



Federated Learning with TensorFlow Federated (TFF)

An open-source framework by Google for simulating and deploying federated learning at scale, built on top of TensorFlow

What is TensorFlow Federated?

A Python framework for **federated computation** that:

- Simulates training across multiple clients
- Works with **TensorFlow models & datasets**
- Provides two core APIs:

`tff.learning`

High-level machine learning API

`tff.federated_computation`

Low-level federated operations



Why Use TensorFlow Federated?



Research Simulation

Test and validate federated learning approaches before deployment in the real world



Ready-to-Use Algorithms

Implementations of popular federated algorithms including FedAvg, FedProx, and others



TensorFlow Integration

Seamlessly works with TensorFlow Keras models and existing ML workflows



Privacy Preservation

Experiment with privacy-preserving methods like differential privacy and secure aggregation

Federated Learning Workflow in TFF

01

Define Global Model

Create a TensorFlow/Keras model that will be shared across all clients

02

Wrap with TFF

Use `tff.learning.from_keras_model` to prepare model for federated training

03

Build Federated Algorithm

Implement `tff.learning.algorithms.build_weighted_fed_avg` for aggregation

04

Run Training Loop

Execute federated training across multiple simulated clients

05

Evaluate Performance

Assess model accuracy and convergence across the federation

Example: Building a Federated Averaging Model

```
import tensorflow as tf
import tensorflow_federated as tff

# Create Keras model
def create_model():
    return tf.keras.Sequential([
        tf.keras.layers.Input(shape=(784,)),
        tf.keras.layers.Dense(128, activation='relu'),
        tf.keras.layers.Dense(10, activation='softmax')
    ])

# Wrap for TFF
def model_fn():
    return tff.learning.from_keras_model(
        create_model(),
        input_spec=tf.TensorSpec([None, 784], tf.float32),
        loss=tf.keras.losses.SparseCategoricalCrossentropy(),
        metrics=[tf.keras.metrics.SparseCategoricalAccuracy()]
    )
```

This code creates a simple neural network for digit classification and wraps it for federated learning

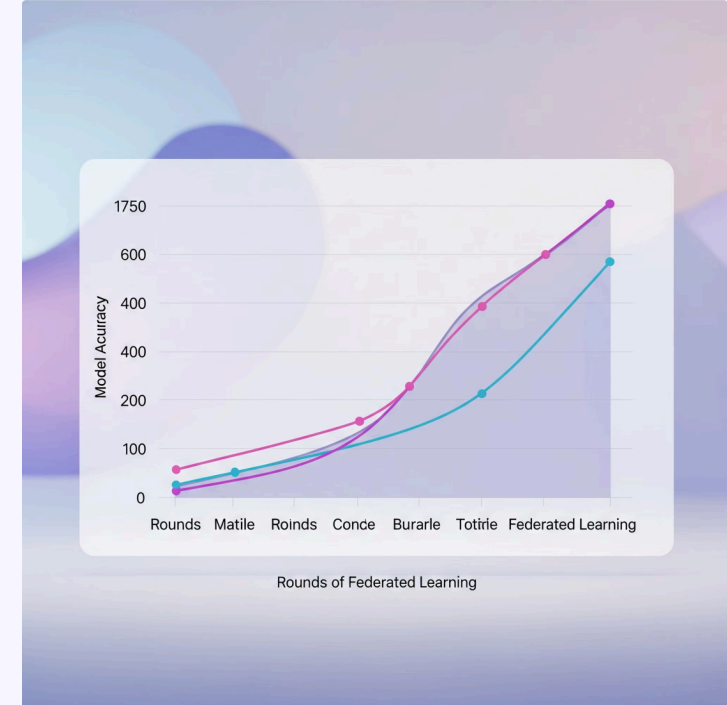
Federated Averaging Implementation in TFF

```
# Build FedAvg algorithm
trainer = tff.learning.algorithms.build_weighted_fed_avg(model_fn)

# Initialize state
state = trainer.initialize()

# Simulate federated training
for round_num in range(5):
    state, metrics = trainer.next(state, federated_data)
    print(f"Round {round_num}, Metrics: {metrics}")
```

✅ **Expected Output:** Accuracy improves over rounds as models aggregate across clients



The FedAvg algorithm coordinates model updates across all participating devices without sharing raw data

Creating Simulated Federated Datasets

```
import tensorflow_federated as tff

emnist_train, mnist_test =
tff.simulation.datasets.emnist.load_data()

# Sample clients (e.g., 5 clients)
sample_clients = mnist_train.client_ids[:5]
federated_data = [
    mnist_train.create_tf_dataset_for_client(c)
    for c in sample_clients
]
```

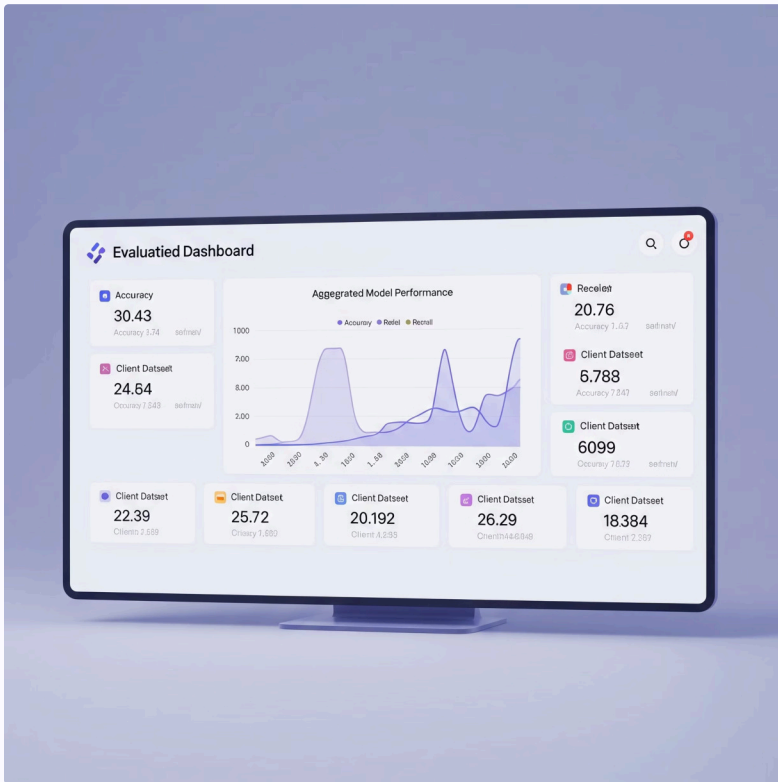
Key Benefits:

Pre-partitioned datasets organized by user/client
(EMNIST, Shakespeare, Stack Overflow)

Perfect for simulating **non-IID data distributions**
(realistic client heterogeneity)

Scale from a few to **thousands** of simulated clients

Model Evaluation in TFF



```
evaluator = tff.learning.algorithms.build_fed_eval(model_fn)
state, metrics = evaluator.next(state, [emnist_test])
print(metrics)
```

Evaluation ensures:

- Model **generalizes beyond client-specific data**
- Performance is consistent across heterogeneous data distributions
- Global model provides accurate metrics for loss and accuracy
- Convergence is tracked across federated learning rounds

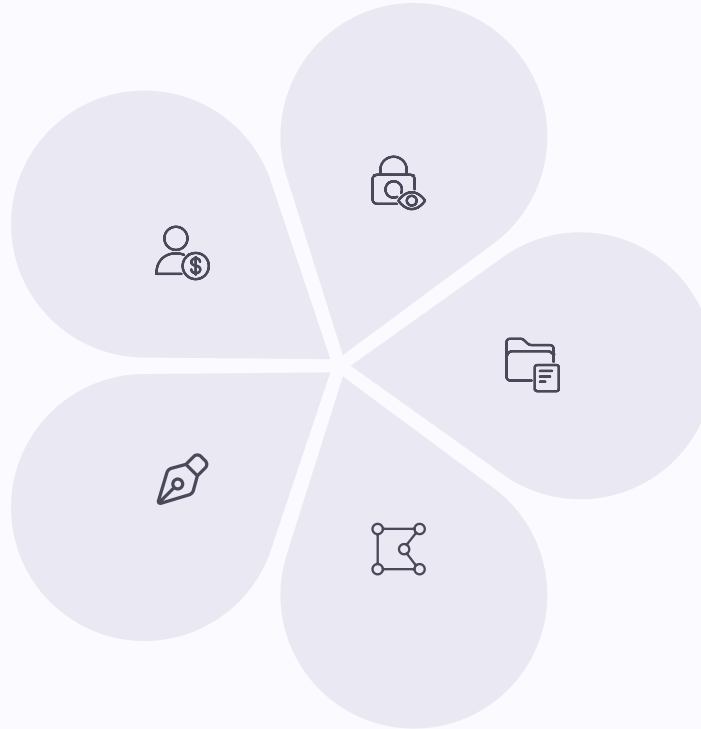
Customizing Federated Learning in TFF

Client Optimization

Modify optimizers per client (SGD, Adam) or add regularization penalties (FedProx)

Model Compression

Reduce communication costs with model quantization and pruning techniques



Differential Privacy

Add noise during aggregation with DP optimizers to protect individual contributions

Secure Aggregation

Implement encryption during transport for additional security guarantees

Custom Aggregation

Build specialized aggregation functions with low-level APIs for research

TFF's flexible architecture allows researchers to implement and test novel federated learning approaches

Key Advantages of TensorFlow Federated

Scalable Simulation

Easily simulate **millions of clients** on standard hardware

Extensible Framework

Open-source architecture designed for research experimentation

Realistic Data Handling

Built to handle **non-IID data** distributions naturally

Production Pathway

Bridge from research concepts to real TensorFlow models



Limitations of TensorFlow Federated

Research-First Design

Not yet production-ready for large-scale deployment

Primarily designed as a simulation environment rather than deployment framework

Hardware Acceleration

Limited GPU/TPU support in simulation mode

Performance bottlenecks when scaling to very large numbers of clients

Use Case Focus

Better suited for **prototyping FL algorithms** than production deployment

May require custom engineering for real-world implementation



Industry Use Cases of TensorFlow Federated

Healthcare Applications

Testing federated methods on healthcare datasets before real-world deployment

Academic Research

Exploring novel **federated optimization algorithms** and convergence properties

Mobile Applications

Training **keyboard prediction models** and other on-device intelligence

Educational Tool

Teaching **privacy-preserving machine learning** concepts in academic settings

Summary: TensorFlow Federated



Simulation Framework

TFF provides a comprehensive environment for federated learning experimentation



Ready-to-Use Tools

Access to **pre-built algorithms and datasets** accelerates research



Research-First Design

Ideal for **research, teaching, and prototyping** federated learning systems



Pathway to Production

Foundation for scaling into **real federated infrastructures**

Start building privacy-preserving ML with TensorFlow Federated today!